

Per-session color design

Revision: 1 **Last modified:** 2026-06-18T22:04:29Z **Status:** design / approved
Authority: CLAUDE.md §102 (operator-path coverage), §103 (four-layer coverage), §11.4.6 (no guessing), §11.4.4(b) (four-layer test floor), §11.4.81 (cross-OS parity), §11.4.93/§11.4.95 (single SSoT), §11.4.151 (release-tag prefix).
Brainstorming-skill terminal flow → writing-plans → subagent-driven-development.

Per-session color via name:color[:params]

1. Goal

Let the operator choose an explicit per-session color at session-creation time by typing the color into the `-s/-t` value, colon-delimited:

```
tmx new -s work          # today's behaviour - hostname-derived color
tmx new -s work:red      # explicit red; persisted; re-used on bare-name re-runs
tmx new -s deploy:#3b82f6 # explicit hex color
tmx new -s work:red:x:y  # color + ignored extra fields (forward-compatible)
```

Today every session on a given host is the *same* (hostname-derived) color. This adds an explicit, per-session, persisted override.

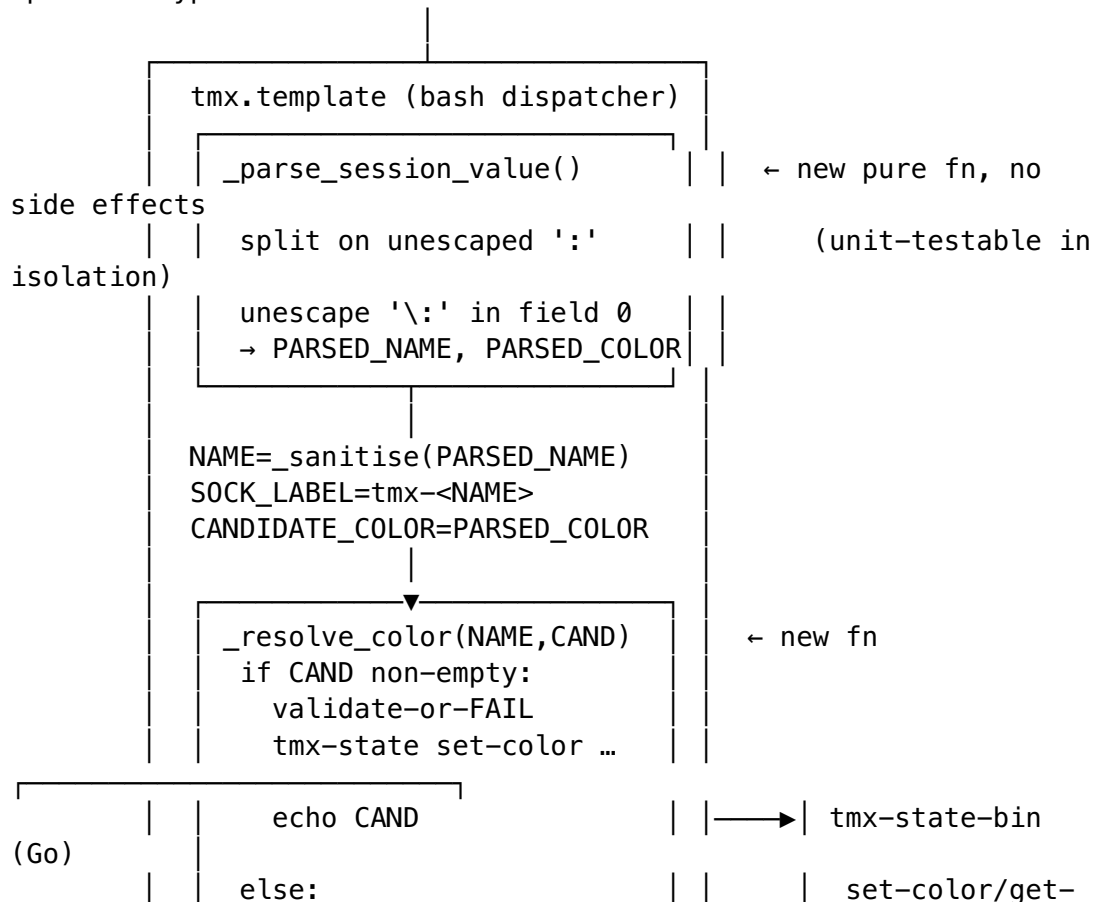
2. Decisions (locked at brainstorm, 2026-06-18)

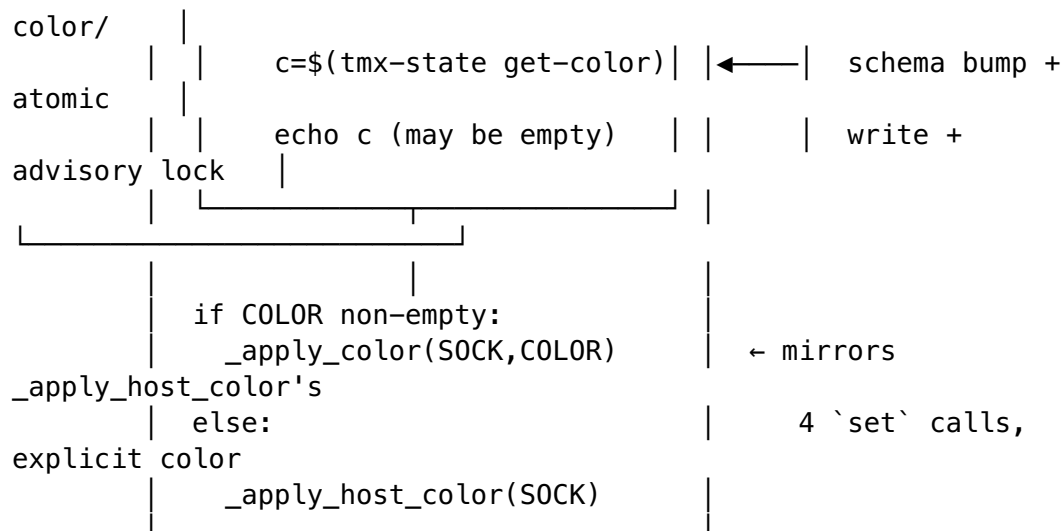
#	Decision	Choice
1	Color scope	All 4 "green" surfaces the existing <code>_apply_host_color()</code> touches: <code>status-style bg</code> , <code>pane-active-border-style fg</code> , <code>clock-mode-colour</code> , <code>window-status-current-style bg</code> . Yellow mode/message styles are deliberately NOT recolored (contrast).
2	Color formats	tmux color names (red, green, yellow, blue, magenta, cyan, white, black, plus <code>colour0-colour255</code>) and <code>#RRGGBB /</code>

#	Decision	Choice
		#RGB hex. Validated — invalid input is a hard error (no bluff, §11.4.6).
3	: in name	A literal <code>:</code> is the field delimiter; a <code>\:</code> <i>inside the name field</i> is unescaped to a literal <code>:</code> . Escapes honored only in field 0.
4	Hostname relation	Explicit color overrides the hostname color for that session and persists the preference.
5	Other params	Extra <code>:</code> fields (beyond name + color) are silently ignored for now. The parser is forward-compatible but ships zero params beyond color.
6	Persistence store	Extend <code>scripts/tmx-state/</code> (the Go binary <code>tmx-state-bin</code> + <code>~/tmx/state.json</code>) with a <code>color</code> field on <code>Session</code> + <code>set-color/get-color</code> subcommands. Reuses the existing atomic-write + advisory-lock + schema-version machinery. Single SSoT for all per-session state (§11.4.93/§11.4.95).
7	Bare-name re-run	When <code>tmx new -s foo</code> is run and <code>foo</code> has a persisted color, the persisted color wins; only if no persisted color exists does the hostname-derived color apply. Precedence: inline color > persisted color > hostname color > default green.

3. Architecture (Approach A — parse in wrapper, validate+store in state binary)

operator types: `tmx new -s work:red`





Units (each one clear purpose, independently testable)

Unit	Language	Responsibility	Depends on
<code>_parse_session_value</code>	bash (wrapper)	split name:color:... on unescaped :, unescape \: in name, emit name + raw color candidate (+ ignored tail). Pure.	nothing
<code>_color_valid</code>	bash (wrapper)	return 0 if token is a known tmux name / colourNNN / valid #hex, else 1 + stderr reason. Pure.	a closed name list + regex
<code>tmx-state set- color / get-color</code>	Go (state binary)	persist/read the per- session color in state.json atomically under the existing lock; set- color validates the token identically.	existing atomic-write + lock infra
<code>_resolve_color</code>	bash (wrapper)	given (NAME, inline candidate): validate+persist+return candidate, else return persisted color, else empty. Orchestrates the other three.	the 3 above
<code>_apply_color</code>	bash (wrapper)	apply an explicit color to the 4 green surfaces via tmux set -g ... on SOCK_LABEL.	TMUX_BIN, socket
<code>_apply_host_color</code>	bash (wrapper, existing)	unchanged — the fallback path when no	hostname_color.sh, socket

Unit	Language	Responsibility	Depends on
		explicit/persisted color exists.	

4. Parsing contract (normative)

Input to `_parse_session_value` is the exact string the operator passed after `-s` or `-t` (one shell arg).

Algorithm (POSIX-portable; must parse under `sh -n` per §11.4.67):

1. Walk the string char by char. Build field 0 until the first **unescaped** `:`. An escaped `\:` contributes a literal `:` to field 0 and does NOT end it. A `\` that is not followed by `:` contributes a literal `\`. Any other char contributes itself.
2. The first unescaped `:` ends field 0 (the name). Everything after it is re-split on unescaped `:` into fields 1..N (escapes are NOT honored in fields ≥ 1 — they pass through literally; this matches decision #3).
3. Field 1 (if present and non-empty) is the **color candidate**. Fields 2..N are ignored for now (decision #5).
4. Output: `PARSED_NAME` (raw, pre-`_sanitise`) and `PARSED_COLOR` (empty string if there was no second field or it was empty).

Then (existing flow, unchanged): `NAME=$( _sanitise "$PARSED_NAME" )`,
`SOCK_LABEL=tmx-{$NAME}`.

Worked examples

operator input (<code>-s</code> value)	PARSED_NAME	PARSED_COLOR	ignored tail
work	work	<i>(empty)</i>	—
work:red	work	red	—
deploy:#3b82f6	deploy	#3b82f6	—
a\:b:cyan	a:b	cyan	— (name sanitises to a_b)
work:red:x:y	work	red	x:y
work:	work	<i>(empty)</i>	— (treated as bare name)

5. Color validation (normative)

`_color_valid <token>` returns 0 (valid) iff token matches one of:

- a tmux named color, **case-insensitive on input**, drawn from this canonical closed set (stored lowercase; the bash list and the Go list MUST be byte-identical copies of this very line — they are the single source of truth, and a divergence between them is the §11.4.6 guessing surface this section exists to close): red green yellow blue magenta cyan white black brightred

- brightgreen brightyellow brightblue brightmagenta brightcyan
brightwhite default terminal; **or**
 - colour / color followed by 1–3 digits, value 0–255 (e.g. colour160, color39); **or**
 - # followed by exactly 3 or 6 hex digits, case-insensitive (#3b82f6, #f0a).
- Validated by POSIX ERE, no colour-name expansion needed at parse time — the wrapper passes a tmux-style value to `tmux set`:
- named → pass verbatim (red).
 - colourNNN → pass verbatim (colour160).
 - #hex → pass as #3b82f6 (tmux ≥ 2.9 + the shipped `terminal-overrides ...:Tc` true-color support render it directly; the conf template already enables Tc).

Anything else → return non-zero + stderr

tmx: invalid color '<token>' (use a tmux name, colour0–255, or #hex).

6. Persistence schema change (Go state binary)

6.1 state.go — add field, bump schema

Session gains one field:

```
type Session struct {
    LastPwd      string `json:"last_pwd"`
    LastSeenUnix int64  `json:"last_seen_unix"`
    CreatedUnix  int64  `json:"created_unix"`
    Host         string `json:"host,omitempty"`
    Color        string `json:"color,omitempty"` // NEW - tmux
                    color token, "" = none
}
```

SchemaVersion is bumped 1 → 2. `loadState` MUST accept `schema_version 1` files (read them fine — the new field is `omitempty`, so old files simply have `Color:""`), and on next write stamp `schema_version 2`. A `schema-2` file read by an old (`schema-1`) binary is also safe (Go JSON `unmarshal` ignores unknown fields). **No migration script needed** — additive `omitempty` field + forward/backward JSON tolerance. (Documented as an honest §11.4.6 non-event, not a risk.)

6.2 main.go — new subcommands

```
tmx-state set-color <session> <color>    # validate + persist; exit
1 on invalid color
tmx-state get-color <session>              # print color (no newline)
+ exit 0; "" + exit 1 if none/unset
```

- `set-color` reuses `withStateLock + loadState + saveStateUnlocked` (identical pattern to `record`). It preserves all other fields (`LastPwd`, `CreatedUnix`, etc.) via read-modify-write. It validates the color token with a `Go validColor()` whose accepted set is the **byte-identical copy of §5's canonical list** (same literals, same `colourNNN` rule, same `#hex` ERE) — bash

- and Go carry matching copies of the one list, and a unit test on each side asserts the same probe tokens agree, so persistence + wrapper can never disagree.
- `get-color` mirrors `recall`'s exit-code contract (0 + value, 1 + "" if not present). State-binary-failure path returns exit 1 (non-fatal in the wrapper — see §8).
- list output gains a 4th TSV column `COLOR` (empty for legacy rows) so `tmx-state list` stays a complete view of persisted state.

6.3 Version string

`const Version = "tmx-state v1.1.0"` — feature bump (additive subcommands + schema field). `scripts/setup.sh` rebuilds `tmx-state-bin`.

7. Wrapper changes (`tmx.template`)

7.1 New helpers (above the dispatch block)

```
# Pure: split "name:color[:ignored]" on unescaped ':', unescape
#       '\:' in name.
# Sets globals PARSED_NAME + PARSED_COLOR. No side effects.
_parse_session_value() { ... }

# Pure: return 0 iff $1 is a valid tmux color token (name /
#       colourNNN / #hex).
_color_valid() { ... }

# Orchestrate: given (NAME, inline candidate) decide the effective
#               color,
# persisting inline candidates. Prints the effective color (may be
#               empty).
_resolve_color() { ... }

# Apply an explicit color to the 4 green surfaces (mirrors
#               _apply_host_color
# minus the hostname derivation).
_apply_color() { # args: sock_label color
    local sock_label="$1" color="$2"
    "$TMUX_BIN" -L "$sock_label" set -g status-style
    "bg=$color" 2>/dev/null || true
    "$TMUX_BIN" -L "$sock_label" set -g pane-active-border-style
    "fg=$color" 2>/dev/null || true
    "$TMUX_BIN" -L "$sock_label" set -g clock-mode-colour
    "$color"    2>/dev/null || true
    "$TMUX_BIN" -L "$sock_label" set -g window-status-current-
    style "bg=$color,fg=black" 2>/dev/null || true
}
```

7.2 Wired into the `arg-parse` → `NAME` flow

After the existing loop that captures `SESSION_ARG/TARGET_ARG`, replace the direct `NAME="$SESSION_ARG"` assignment with:

```

RAW_NAME=""
[ -n "$SESSION_ARG" ] && RAW_NAME="$SESSION_ARG"
[ -z "$RAW_NAME" ] && [ -n "$TARGET_ARG" ] &&
    RAW_NAME="$TARGET_ARG"
[ -z "$RAW_NAME" ] && RAW_NAME="default"

_parse_session_value "$RAW_NAME"
NAME=$( _sanitise "$PARSED_NAME" )
# resolve color AFTER we know NAME (uses tmx-state-bin; non-fatal
  if absent)
EFFECTIVE_COLOR=$( _resolve_color "$NAME" "$PARSED_COLOR" || true)

```

If `_resolve_color` reports an **invalid inline color** (exit non-zero with a distinct code, e.g. 5), the wrapper prints the reason to `stderr` and exits **before** creating any socket/scope/session (fail-fast, §11.4.6 — do not create a session with a silently-dropped color).

7.3 Applied in the new branch

In the `new|new-session|start-server|""` case, after the per-OS isolation block + after `_apply_oom_score`, replace the unconditional `_apply_host_color "$SOCK_LABEL"` with:

```

if [ -n "$EFFECTIVE_COLOR" ]; then
    _apply_color "$SOCK_LABEL" "$EFFECTIVE_COLOR"
else
    _apply_host_color "$SOCK_LABEL"
fi

```

7.4 Re-apply on attach (consistency)

The existing attach branch already re-source-files the conf into a pre-existing session. Add a parallel re-apply: if `EFFECTIVE_COLOR` is non-empty, `_apply_color` on attach too, so re-attaching an old session reflects its persisted color. Non-fatal + silent on failure (same posture as the existing attach reload).

7.5 macOS bridge

No change. `tmx-mac.template` forwards the `-s` value verbatim as one shell-quoted arg; `tmx.template` parses it VM-side. §11.4.81 parity holds by construction.

8. Error handling & precedence

`EFFECTIVE_COLOR` resolution:

```

1. inline PARSED_COLOR non-empty?
    validate → invalid? → stderr + exit 5 (no session
created)
                                valid? → tmx-state set-color NAME COLOR
(best-effort;
                                failure is logged, non-fatal) →
EFFECTIVE=COLOR

```

- ```
2. else: tmx-state get-color NAME
 success? → EFFECTIVE=persisted
 absent/fail? → EFFECTIVE="" → hostname color path
```
- State-binary unavailable / unreadable: NON-FATAL (the wrapper's "tmx-never-breaks" invariant is preserved — same posture as the existing cwd-recall fallback to \$HOME). Inline color still applies for the current run; persistence just doesn't survive. Logged to stderr at most once.
  - Collision checks (Linux scope / Darwin socket) happen **before** color resolution and are unchanged.
  - tmx kill-session does NOT clear the persisted color (a re-create should honor the preference — decision #7). tmx-state forget (existing) removes the whole session record including color. A dedicated unset-color is YAGNI for now (the name: form re-sets; forget clears).

## 9. Testing — four-layer coverage (§11.4.4(b), §102 operator-path)

Every test drives the **real operator path** (`tmx new -s ...`), never a hand-spawned equivalent. Anti-bluff: every PASS cites captured runtime evidence (a tmux option value read back from the live server via `tmux -L <sock> show-options -gv status-style`), never an exit code alone.

### Layer 1 — pre-build gate (`scripts/tests/NN_session_color.sh, new`)

Greps the wrapper + state sources for the contract invariants and refuses to build if missing (paired §1.1 mutation: strip one → gate FAILs). Asserts:

- `_parse_session_value + _color_valid + _resolve_color + _apply_color` exist in `tmx.template`.
- `set-color / get-color` subcommands exist in `tmx-state/main.go`.
- `Session.Color` field + `SchemaVersion==2` in `state.go`.

### Layer 2 — unit (Go) `scripts/tmx-state/*_test.go`

- `state_test.go`: round-trip `Color` field; schema-1 file loads with `Color==""`; schema-2 stamps on save; list 4th column present.
- `main_test.go`: `set-color` rejects invalid tokens (exit 1); `get-color` exit-code contract (0/1); `set-color` preserves sibling fields.
- New `_parse_test.go`-equivalent for the bash parser lives in Layer 3.

### Layer 3 — runtime operator-path tests (new `scripts/tests/` files)

Driven through `tmx new -s ...` against the real built wrapper + state binary. Each captures the live `status-style` from the socket as positive evidence:

| Test                | Operator path | Asserted evidence |
|---------------------|---------------|-------------------|
| NN_name_color_basic |               |                   |



| Test                            | Operator path                                       | Asserted evidence                                                                                                                        |
|---------------------------------|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
|                                 | tmx new -s<br>work:red -d                           | tmux -L tmx-work show<br>-gv status-style ==<br>bg=red                                                                                   |
| NN_name_color_hex               | tmx new -s<br>deploy:#3b82f6<br>-d                  | status-style ==<br>bg=#3b82f6                                                                                                            |
| NN_name_color_persist           | set work:red; kill;<br>tmx new -s work<br>-d (bare) | status-style == bg=red<br>(persisted wins)                                                                                               |
| NN_name_color_invalid           | tmx new -s<br>work:notacolor                        | exit non-zero; <b>no</b> socket<br>tmx-work created                                                                                      |
| NN_name_color_escape            | tmx new -s<br>'a\:b:cyan' -d                        | socket tmx-a_b exists;<br>status-style ==<br>bg=cyan                                                                                     |
| NN_name_color_extra_ignored     | tmx new -s<br>work:red:x:y -d                       | status-style == bg=red;<br>no error                                                                                                      |
| NN_name_color_all4_surfaces     | tmx new -s<br>work:blue -d                          | all 4 options (status-<br>style/pane-active-<br>border-style/clock-<br>mode-colour/window-<br>status-current-style)<br>read back == blue |
| NN_name_color_hostname_fallback | tmx new -s<br>fresh -d (no<br>persisted color)      | status-style ==<br>bg=<hostname_color.sh<br>output> (fallback path<br>intact)                                                            |

All DETERMINISTIC per §11.4.50 (N=3 identical). Cleanup: `tmux kill-server` on the test sockets in a trap.

## Layer 4 — HelixQA Challenge + paired §1.1 mutation

- HelixQA Challenge (in `scripts/challenges/tmux.yaml`): a scenario that creates a colored session, reads back all 4 surfaces, and scores PASS only on captured evidence the explicit color landed (not green, not hostname color).
- Paired mutation (meta-test): strip `_apply_color` from the wrapper → the runtime test FAILs (color does not apply). Strip `set-color` → persistence test FAILs. This proves the tests genuinely catch the defect class (§1.1).

## Layer 0 — bash parseability (§11.4.67)

`sh -n scripts/tmux.conf.template + bash -n scripts/tmx.template` after every edit. The new pure functions are POSIX-portable (no bash-only constructs unguarded).

## 10. Documentation updates (same commit, §11.4.12/§11.4.65)

- docs/guide/README.md — new "Per-session color" subsection with the operator-path examples from §3 + the escape rule.
- CLAUDE.md Commands table — extend the `tmx {new|attach|ls|kill}` row note, or add a one-line "session value may be name:color".
- Fixed.md / Issues.md — migrate a new ATM-NNN item (opened then closed in the same cycle).
- Version bump: `VERSION version= + versionCode=` together with a `CHANGELOG.md` entry; release tag prefixed per §11.4.151 (`tmux-<version>` — prefix resolved from `HELIX_RELEASE_PREFIX` or the lowercased root dir name).

## 11. Cross-OS parity (§11.4.81)

- Linux: full path — wrapper parses, state binary persists, tmux applies.
- macOS: the bridge forwards the `-s` value verbatim; the VM-side wrapper does all the work. No Darwin-specific branch needed.
- Honest gap: none for this feature. (Hostname-color's XNU `RLIMIT_AS` gap is unrelated and already documented.)

## 12. Out of scope (YAGNI)

- Extra `:params` beyond color (decision #5).
- `unset-color` subcommand (use `tmx-state forget` or `re-set`).
- Recoloring the yellow mode/message styles (decision #1).
- A TUI color picker.
- Applying color to the macOS host's native tmux (out of scope — tmx owns the colored path; system tmux is untouched by design).

## 13. Acceptance criteria (definition of done)

All of:

1. `tmx new -s NAME:COLOR` creates a session whose **live** `status-style` (and the other 3 surfaces) read back as the chosen color — captured evidence, not exit code.
2. Re-running `tmx new -s NAME` (bare) re-uses the persisted color.
3. An invalid color is rejected before any session/socket is created.
4. `tmx-state get-color NAME` returns the persisted value; schema is v2.
5. All four test layers GREEN; paired mutation FAILs when the feature is stripped; HelixQA Challenge PASSes on captured evidence.
6. Linux + macOS both work (macOS via the bridge, verified by the existing bridge-forward semantics + at least one macOS-path runtime assertion if a Mac is available, else documented §11.4.3 SKIP-with-reason).
7. Docs + version + changelog updated in the same commit; release tag prefixed per §11.4.151.